

Cybersecurity Project Presentation

# Keylogger & Countermeasure Demonstrator

A full offensive and defensive study analyzing hardware input interception  
methods and safe software countermeasures.

---

Navpreet Kaur Dhillon<sup>r</sup>

Sushanti Anam<sup>r</sup>

Sadia Naseer<sup>r</sup>

Vuzumuzi Zondo

# Basic Definitions

## The Keylogger

A keylogger is a tool that quietly records every button you type on your physical keyboard. Hackers use them silently to copy passwords, bank details, and personal texts without you knowing.

## The Countermeasure

A countermeasure is an action or utility used to block a computer attack. In this project, our countermeasure is an on-screen keyboard that stops spy scripts from seeing what you type.

# The Problem: Silent Data Theft

-  **Physical Typing Vulnerability:** Operating systems pass physical typing signals globally. This allows simple background scripts to copy what you write.
-  **Invisibility:** Keylogger processes run without any window. A regular computer user has no idea that their keyboard is being tracked.
-  **Automatic Logging:** Stolen keystrokes can be saved instantly and sent straight to an online panel where hackers read them in real-time.
-  **Security Gap:** Standard security tools can miss simple customized typing scripts, meaning users need visual counters to stay safe.

# Data Storage: database.py

database.py

## The Project's Storage Center

This python module sets up a secure, local **SQLite3 database** called logs.db. It acts as the local memory for our keylogger. Its tasks are simple and clear:

- Sets up a database table with columns for id, timestamp, and keys.
- Offers a save\_key() command that instantly writes a single keypress.
- Offers a get\_all\_keys() command to pull the saved data.



# Attacker Dashboard: app.py

app.py

## Visualizing the Threat

This module uses the **Flask web framework** to build a visual, real-time control page. It shows what a bad actor can see remotely.

First, it boots a background listener thread using the `pynput.keyboard` module. This captures standard typing events silently across the entire computer.

The web panel loads these events from the database and automatically refreshes every 3 seconds, displaying newly typed characters instantly.



# The Secure Shield: defence.py



defence.py

## Secure Virtual Typing Panel

This file builds a custom **Virtual Keyboard** using Python's simple **Tkinter** library. It allows you to enter text using mouse clicks instead of your physical keyboard.

**Why it works:** The keylogger tracks physical hardware keyboard signals. Because you are clicking visual buttons on the screen, no physical key signals are sent. The logger registers nothing, keeping your passwords safe!

# Keylogger and Countermeasure Demonstrator

## Intercepted Keystrokes (Database Logs)

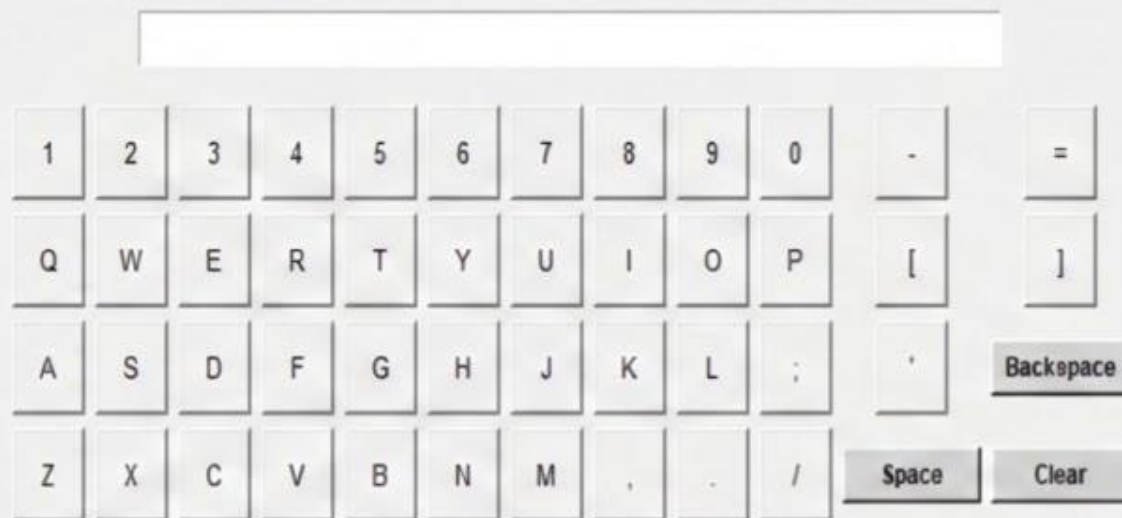
This web panel simulates what an attacker sees remotely.

| Timestamp           | Key Captured |
|---------------------|--------------|
| 2026-07-07 21:09:53 | [backspace]  |
| 2026-07-07 21:09:49 | [backspace]  |
| 2026-07-07 21:09:46 | □            |
| 2026-07-07 21:09:45 | [ctrl_l]     |
| 2026-07-07 21:09:07 | [delete]     |
| 2026-07-07 21:08:21 | [backspace]  |
| 2026-07-07 21:08:11 | [backspace]  |

### The database for Keylogger

It stores all the keys pressed on the keyboard

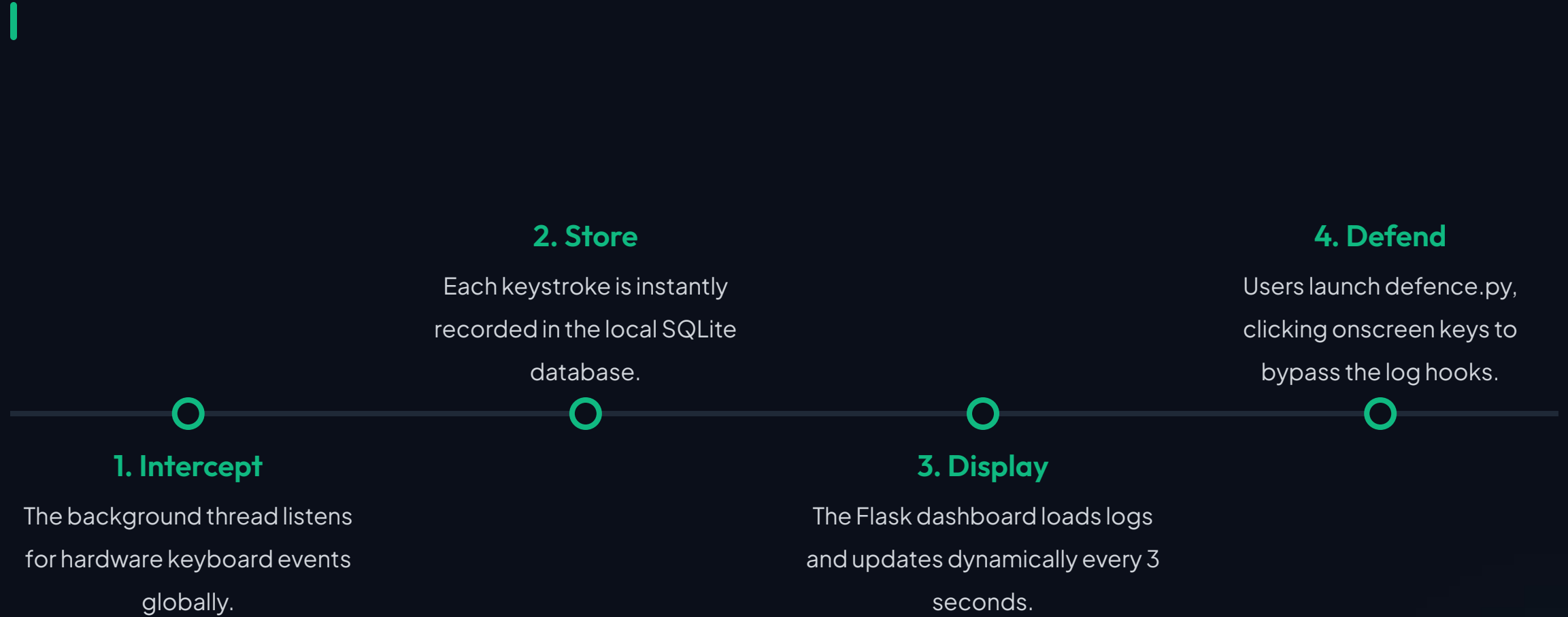
## Countermeasure: Virtual Typing Panel (Bypasses pynput hooks)



### The countermeasure

The countermeasure is an on-screen keyboard that stops spy scripts from seeing what you type.

# Project Operational Steps



## 1. Intercept

The background thread listens for hardware keyboard events globally.

## 2. Store

Each keystroke is instantly recorded in the local SQLite database.

## 3. Display

The Flask dashboard loads logs and updates dynamically every 3 seconds.

## 4. Defend

Users launch defence.py, clicking onscreen keys to bypass the log hooks.



# How to Run & Test the Project



## 1. Prep Files

Ensure you have all three python files: app.py, database.py, and defence.py in the same folder. Install the dependencies using pip:

```
pip install flask pynput
```



## 2. Start Server

Launch the logger server using your terminal. This will automatically set up the storage database and start tracking keystrokes:

```
python app.py
```



## 3. Launch Shield

Open another terminal window and start the visual secure board. Try typing both on your hardware keys and the screen panel to watch the

bypass in action!

```
python defence.py
```